

Task 1: To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop

```
saikrishna@DESKTOP-LGFVDTJ:~$ nano task1.c
saikrishna@DESKTOP-LGFVDTJ:~$ gcc task1.c -o task1
saikrishna@DESKTOP-LGFVDTJ:~$ ./task1
my_shell> is
You entered: is
my_shell> Is
You entered: Is
my_shell> sh
You entered: sh
my_shell> gcc main.c -o my_shell
You entered: gcc main.c -o my_shell
my_shell> helloooooo aklhilll
You entered: helloooooo aklhilll
my_shell> cat task1.c
You entered: cat task1.c
my_shell> cat task1.c
You entered: cat task1.c
my_shell> exit
Exiting shell...
saikrishna@DESKTOP-LGFVDTJ:~$ cat task1.c
#include <stdio.h>
#include <string.h>

int main()
{
    char command[100];

    while (1)
    {
        printf("my_shell> ");

        fgets(command, sizeof(command), stdin);

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
        {
            printf("Exiting shell...\n");
            break;
        }

        printf("You entered: %s\n", command);
    }

    return 0;
}
saikrishna@DESKTOP-LGFVDTJ:~$ |
```

Task 2: Using the following system calls, copy the content from File1.txt to File2.txt.

1. Open()
2. Read()
3. Write()
4. Close()

open()

```
saikrishna@DESKTOP-LGFVDTJ:~$ cat task2.c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main()
{
    int source, destination;
    char buffer[1024];
    ssize_t bytesRead, bytesWritten;

    source = open("File1.txt", O_RDONLY);

    if (source < 0)
    {
        perror("Error opening File1.txt");
        return 1;
    }

    destination = open("File2.txt",
                      O_WRONLY | O_CREAT | O_TRUNC,
                      0644);

    if (destination < 0)
    {
        perror("Error opening File2.txt");
        close(source);
        return 1;
    }

    while ((bytesRead = read(source, buffer, sizeof(buffer))) > 0)
    {
        bytesWritten = write(destination, buffer, bytesRead);

        if (bytesWritten != bytesRead)
        {
            perror("Error writing file");
            close(source);
            close(destination);
            return 1;
        }
    }

    close(source);
    close(destination);

    printf("File copied successfully.\n");

    return 0;
}

saikrishna@DESKTOP-LGFVDTJ:~$ ./task2
Error opening File1.txt: No such file or directory
saikrishna@DESKTOP-LGFVDTJ:~$ ls
hello.c  OSSP  Practical  Practical.c  hello  hello.c  sai.c  task1  task1.c  task2  task2.c
saikrishna@DESKTOP-LGFVDTJ:~$ echo "This is the content of File1." > File1.txt
saikrishna@DESKTOP-LGFVDTJ:~$ cat File1.txt
This is the content of File1.
```

syntax: open(" filename.txt", modes, permissons)

Example: open("First_Prgm.txt", O_CREAT | O_WRONLY, 0644);

Table 1: Modes of Open system call

Flag	Purpose
O_RDONLY	Open for read only

Flag	Purpose
O_WRONLY	Open for write only
O_RDWR	Open for both read and write
O_CREAT	Create file if it does not exist
O_TRUNC	Truncate (empty) an existing file
O_APPEND	Append data to the end of the file
O_EXCL	Fail if file already exists (used with O_CREAT)

read()

Syntax: read(int fd, void *buf, size_t count);

Example: read(fd, buffer, 100)

#include <stdio.h>

#include <unistd.h>

int main()

{

char buffer[100];

int n;

printf("Enter some text: ");

n = read(0, buffer, sizeof(buffer) - 1);

buffer[n] = '\0'; // Null-terminate the string

printf("You entered: %s", buffer);

return 0;

}

write()

Syntax: write(int fd, const void *buf, size_t count);

Example: write(fd, message, 11);

#include <unistd.h>

int main()

```

{
    write(1, "Hello World\n", 12);
    return 0;
}

```

Task 3: To Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction.

```

vikrishna@DESKTOP-LGFVDTJ:~$ ./task2
file copied successfully.
vikrishna@DESKTOP-LGFVDTJ:~$ gcc task2.c -o task2
vikrishna@DESKTOP-LGFVDTJ:~$ ./task2
file copied successfully.
vikrishna@DESKTOP-LGFVDTJ:~$ nano task3.c
vikrishna@DESKTOP-LGFVDTJ:~$ gcc task3.c task3\
gcc task3.c task3
usr/bin/x86_64-linux-gnu-ld.bfd: cannot find task3gcc: No such file or directory
usr/bin/x86_64-linux-gnu-ld.bfd: /tmp/ccSFPwB0.o: in function 'main':
task3.c:(.text+0x0): multiple definition of 'main'; /tmp/cc15X6Hq.o:task3.c:(.text+0x0): first defined here
usr/bin/x86_64-linux-gnu-ld.bfd: cannot find task3: No such file or directory
collect2: error: ld returned 1 exit status
vikrishna@DESKTOP-LGFVDTJ:~$ gcc task3.c -o task3
vikrishna@DESKTOP-LGFVDTJ:~$ ./task3
/_shell> sh

command: sh
vikrishna@DESKTOP-LGFVDTJ:~$ cat task3.c
#include <stdio.h>

int main()

{
    char buffer[100];
    int index = 0;
    char ch;

    printf("my_shell> ");

    while (1)
    {
        ch = getchar();

        if (ch == '\n')
        {
            buffer[index] = '\0';

            printf("\nCommand: %s\n", buffer);
            break;
        }

        if (ch == '\b')
        {
            if (index > 0)
            {
                index--;
            }
        }
        else if (index < 99)
        {
            buffer[index++] = ch;
        }
    }

    return 0;
}

```

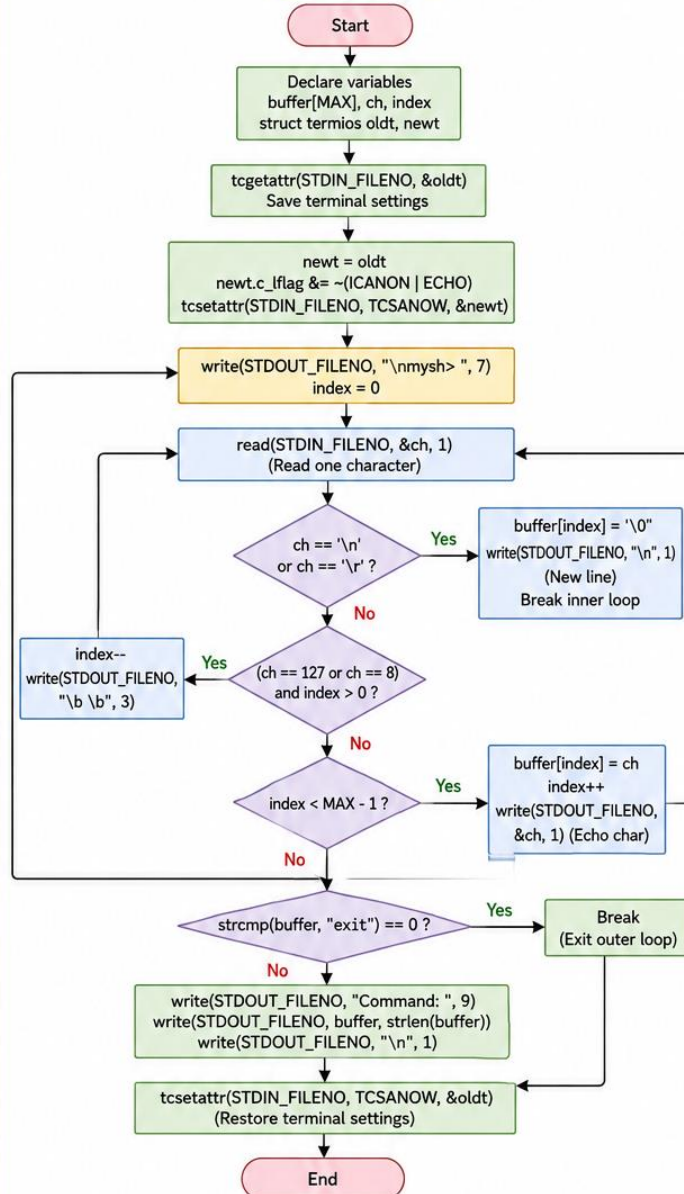
ALGORITHM & FLOWCHART

Keyboard Input Handling Using System Calls (read, write, termios)

ALGORITHM

1. Start.
2. Declare variables:
 - buffer[MAX]
 - char ch
 - int index
 - struct termios oldt, newt
3. Save the current terminal settings using `tcgetattr(STDIN_FILENO, &oldt)`.
4. Copy the terminal settings:
`newt = oldt`
5. Disable canonical mode and echo:
`newt.c_lflag &= ~(ICANON | ECHO)`
6. Apply the new terminal settings using `tcsetattr(STDIN_FILENO, TCSANOW, &newt)`.
7. Repeat forever
 - (a) Display the prompt using `write()`:
"mysh> "
 - (b) Set index = 0.
 - (c) Read characters one by one using `read()`.
 - (d) If the character is Enter ('\n' or '\r'):
 - `buffer[index] = '\0'`
 - Print new line.
 - Exit the inner loop.
 - (e) Else if the character is Backspace (ASCII 8 or 127) and index > 0:
 - `index--`
 - Print "\b \b" to remove last character.
 - (f) Else if index < MAX - 1:
 - Store the character in buffer.
 - `index++`
 - Echo the character using `write()`.
 - (g) After exiting the inner loop, compare the command in buffer.
 - If command is "exit", break the outer loop.
 - Otherwise, print "Command: " followed by the command.
8. Restore the original terminal settings using `tcsetattr(STDIN_FILENO, TCSANOW, &oldt)`.
9. End.

FLOWCHART



SYSTEM CALLS / FUNCTIONS USED

Function / System Call	Purpose
<code>read()</code>	Reads one character from keyboard (stdin).
<code>write()</code>	Writes output to terminal (stdout).
<code>tcgetattr()</code>	Gets current terminal attributes.
<code>tcsetattr()</code>	Sets terminal attributes (raw / canonical mode).
<code>strcmp()</code>	Compares strings (used to check "exit").

TERMINAL MODE

- Canonical mode (default): Input is line buffered. Backspace is handled by terminal driver.
- Non-canonical (raw) mode: Input is available character by character; program handles Backspace, Enter, etc.
- We disable ICANON (canonical) and ECHO to read characters immediately and handle them in the program.

This program uses system calls (read, write) and terminal control (termios) to capture keyboard input, handle backspace, process enter key, manage input buffer and support multi-character commands.

Task 4: To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality

```
saikrishna@DESKTOP-LGFVDTJ:~$ nano task4.c
saikrishna@DESKTOP-LGFVDTJ:~$ nano task4.c
saikrishna@DESKTOP-LGFVDTJ:~$ gcc task4.c -o task4
saikrishna@DESKTOP-LGFVDTJ:~$ ./task4
Buffer content: Hello, Operating Systems!
Memory released successfully.
saikrishna@DESKTOP-LGFVDTJ:~$ cat task4.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main()
{
    char *buffer;

    buffer = malloc(100 * sizeof(char));

    if (buffer == NULL)
    {
        printf("Memory allocation failed\n");
        return 1;
    }

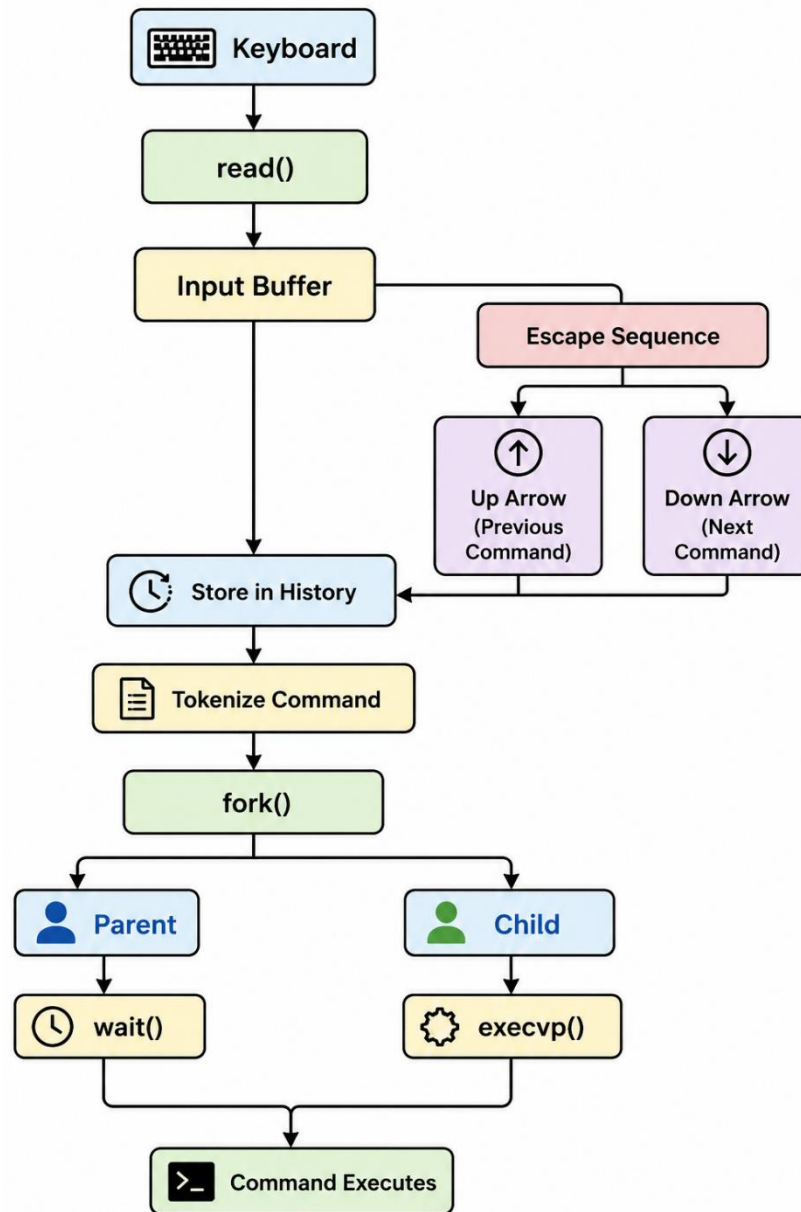
    strcpy(buffer, "Hello, Operating Systems!");

    printf("Buffer content: %s\n", buffer);

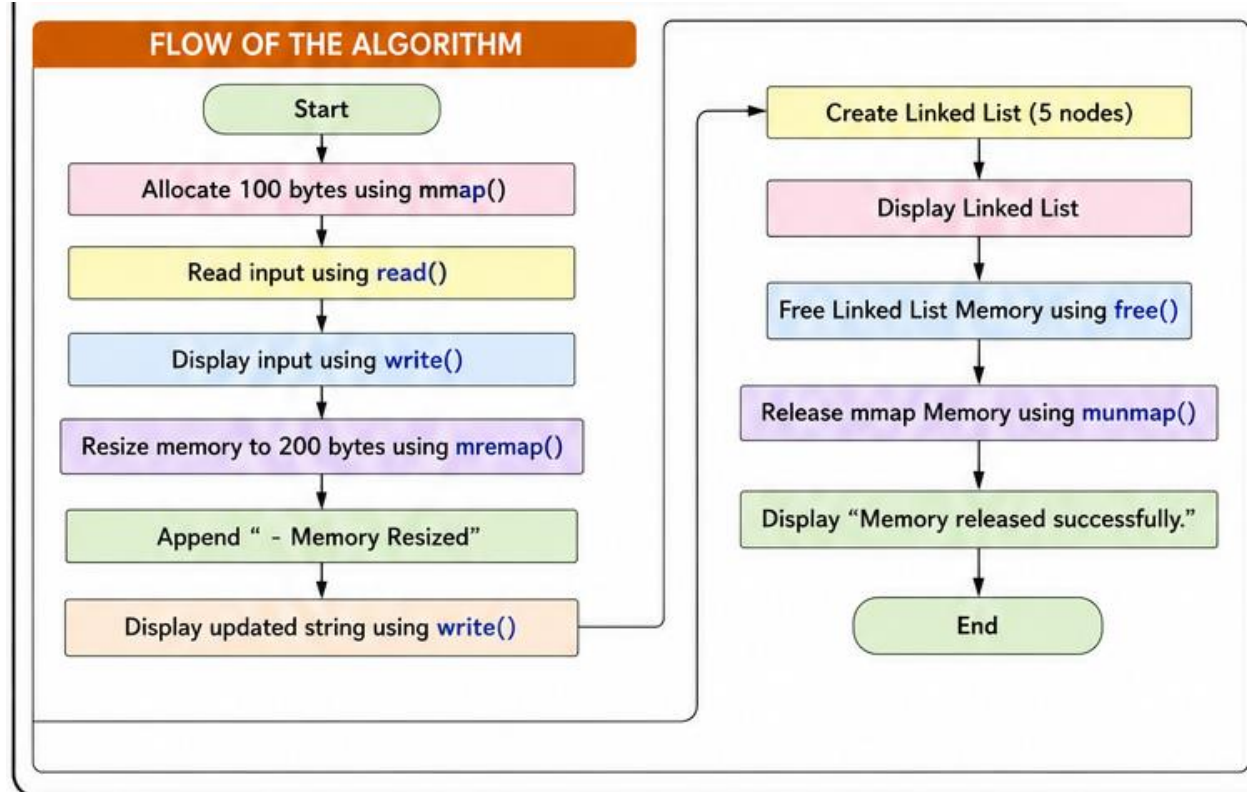
    free(buffer);

    printf("Memory released successfully.\n");

    return 0;
}
saikrishna@DESKTOP-LGFVDTJ:~$ |
```



Task 5: To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind.



Validate the program with valgrind tool